31.4 遗留配置迁移

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~210k tokens，输出 ~8k tokens（本节）

OpenClaw 并非从零诞生——它经历了 moldbot → moltbot → clawdbot → openclaw 的多次更名与架构重组，配置结构也随之不断演进。早期的顶级 `whatsapp` 、 `telegram` 配置键被收编到 `channels.whatsapp` 、 `channels.telegram` ；早期的 `agent.model` 字符串被拆分为 `agents.defaults.model.primary` 加 `fallbacks` 数组； `routing` 下的各种子配置被分散到 `agents` 、 `tools` 、 `messages` 等更语义化的位置。为了让老用户无痛升级，OpenClaw 在配置加载流水线中内置了一套自动迁移系统——本节剖析其设计与实现。

31.4.1 迁移框架（ `src/config/legacy.ts` ）

两个独立的子系统

遗留配置处理由两个互补的子系统组成：

子系统	函数	用途
检测（Detection）	`findLegacyConfigIssues`	扫描配置，发现使用了旧格式的路径， 返回人类可读的警告消息
迁移（Migration）	`applyLegacyMigrations`	自动将旧格式配置转换为新格式，返回变更日志

两者可以独立使用：检测用于在 Web 控制台或 CLI 中显示升级提示；迁移则在配置加载流水线中自动执行。

检测：findLegacyConfigIssues

检测函数遍历一组预定义的规则（ `LEGACY_CONFIG_RULES` ），对每条规则的路径进行查找。如果对应路径存在值且（可选的）匹配条件成立，就记录一条问题：

```typescript
// src/config/legacy.ts

export function findLegacyConfigIssues(raw: unknown): LegacyConfigIssue[] {
  if (!raw || typeof raw !== "object") return [];

  const root = raw as Record<string, unknown>;
  const issues: LegacyConfigIssue[] = [];

  for (const rule of LEGACY_CONFIG_RULES) {
    // 逐层遍历路径
    let cursor: unknown = root;
    for (const key of rule.path) {
      if (!cursor || typeof cursor !== "object") {
        cursor = undefined;
        break;
      }
      cursor = (cursor as Record<string, unknown>)[key];
    }

    // 路径存在且匹配条件满足 → 记录问题
    if (cursor !== undefined && (!rule.match || rule.match(cursor, root))) {
      issues.push({ path: rule.path.join("."), message: rule.message });
    }
  }
  return issues;
}
```

迁移：applyLegacyMigrations

迁移函数使用 `structuredClone` 深拷贝原始配置，然后依次执行所有迁移规则：

```ts
// src/config/legacy.ts

export function applyLegacyMigrations(raw: unknown): {
  next: Record<string, unknown> | null;
  changes: string[];
} {
  if (!raw || typeof raw !== "object") return { next: null, changes: [] };

  const next = structuredClone(raw) as Record<string, unknown>;
  const changes: string[] = [];

  for (const migration of LEGACY_CONFIG_MIGRATIONS) {
    migration.apply(next, changes);
  }

  if (changes.length === 0) return { next: null, changes: [] };
  return { next, changes };
}
```

> **衍生解释**： `structuredClone` 是浏览器和 Node.js（v17+）原生提供的深拷贝 API，比 `JSON.parse(JSON.stringify(obj))` 更强大——它能正确处理 `Date` 、 `RegExp` 、 `Map` 、 `Set` 、循环引用等类型。在这里使用深拷贝至关重要：迁移函数会直接修改对象结构（删除键、移动值），如果不拷贝就会破坏原始输入。

核心类型定义

检测规则和迁移规则的类型定义在 `legacy.shared.ts` 中：

```ts
// src/config/legacy.shared.ts

export type LegacyConfigRule = {
  path: string[];                                          // 要检查的配置路径
  message: string;                                         // 人类可读的提示消息
  match?: (value: unknown, root: Record<string, unknown>) => boolean;  // 可选的匹配条件
};

export type LegacyConfigMigration = {
  id: string;        // 迁移 ID（唯一标识，用于调试）
  describe: string;  // 迁移描述
  apply: (raw: Record<string, unknown>, changes: string[]) => void;  // 执行迁移
};
```

`match` 是一个可选的谓词函数，用于区分同一路径下不同类型值的处理方式。例如 `agent.model` 既可以是字符串（旧格式）也可以是对象（新格式），只有字符串值才需要迁移：

```
{
  path: ["agent", "model"],
  message: "agent.model string was replaced by agents.defaults.model.primary/fallbacks...",
  match: (value) => typeof value === "string",  // 仅匹配字符串值
}
```

入口函数：migrateLegacyConfig

`legacy-migrate.ts` 提供了完整的迁移入口——迁移后立即验证配置有效性：

```
// src/config/legacy-migrate.ts

export function migrateLegacyConfig(raw: unknown): {
  config: OpenClawConfig | null;
  changes: string[];
} {
  const { next, changes } = applyLegacyMigrations(raw);
  if (!next) return { config: null, changes: [] };

  // 迁移后重新验证
  const validated = validateConfigObjectWithPlugins(next);
  if (!validated.ok) {
    changes.push("Migration applied, but config still invalid; fix remaining issues manually.");
    return { config: null, changes };
  }
  return { config: validated.config, changes };
}
```

这个"迁移 + 验证"的组合确保了迁移结果的正确性——如果自动迁移无法完全修复配置（比如涉及到需要人工决策的情况），函数会返回 `null` 并附上提示信息。

辅助工具集

`legacy.shared.ts` 还提供了一组实用的辅助函数，供迁移规则使用：

```typescript
// src/config/legacy.shared.ts（简化）

// 类型守卫：是否为普通对象
export const isRecord = (value: unknown): value is Record<string, unknown> =>
  Boolean(value && typeof value === "object" && !Array.isArray(value));

// 安全获取对象值
export const getRecord = (value: unknown): Record<string, unknown> | null =>
  isRecord(value) ? value : null;

// 确保某个键对应一个对象（不存在则创建空对象）
export const ensureRecord = (root: Record<string, unknown>, key: string) => {
  const existing = root[key];
  if (isRecord(existing)) return existing;
  const next: Record<string, unknown> = {};
  root[key] = next;
  return next;
};

// 将 source 中的键合并到 target 中（不覆盖已有值）
export const mergeMissing = (target: Record<string, unknown>, source: Record<string, unknown>) => {
  for (const [key, value] of Object.entries(source)) {
    if (value === undefined) continue;
    const existing = target[key];
    if (existing === undefined) {
      target[key] = value;
    } else if (isRecord(existing) && isRecord(value)) {
      mergeMissing(existing, value);  // 递归合并
    }
    // 已有值且非对象 → 保留目标值（不覆盖）
  }
};
```

`mergeMissing` 的语义是"只填补缺失"——它不会覆盖目标中已有的值。这在迁移中很重要：用户可能同时拥有新旧两种格式的配置（比如既有 `whatsapp` 又有 `channels.whatsapp`），`mergeMissing` 确保新格式的值不会被旧格式覆盖。

31.4.2 迁移规则（`src/config/legacy.rules.ts`）

检测规则一览

`LEGACY_CONFIG_RULES` 数组定义了 25+ 条检测规则，覆盖了 OpenClaw 历史上所有重大的配置结构变更。按类别分组如下：

渠道迁移（7 条）：

旧路径	新路径	说明
`whatsapp`	`channels.whatsapp`	顶级渠道键 → 统一到 channels 下
`telegram`	`channels.telegram`	同上
`discord`	`channels.discord`	同上
`slack`	`channels.slack`	同上
`signal`	`channels.signal`	同上
`imessage`	`channels.imessage`	同上
`msteams`	`channels.msteams`	同上

路由拆解（8 条）：

旧路径	新路径	说明
`routing.allowFrom`	`channels.whatsapp.allowFrom`	白名单归属到具体渠道
`routing.bindings`	`bindings` （顶级）	绑定关系提升为顶级配置
`routing.agents`	`agents.list`	Agent 列表独立管理
`routing.defaultAgentId`	`agents.list[].default`	默认 Agent 标记在列表项中
`routing.agentToAgent`	`tools.agentToAgent`	Agent 间通信归属到工具
`routing.groupChat.requireMention`	`channels.*.groups."*".requireMention`	群聊配置下沉到渠道级别
`routing.groupChat.mentionPatterns`	`messages.groupChat.mentionPatterns`	提及模式移到消息配置
`routing.queue`	`messages.queue`	消息队列移到消息配置

Agent 配置重组（7 条）：

旧路径	新路径	说明
`agent.*`	`agents.defaults.*`	单数形式 → 复数 +defaults
`agent.model` （字符串）	`agents.defaults.model.primary/fallbacks`	单模型 → 主+备选
`agent.imageModel` （字符串）	`agents.defaults.imageModel.primary/fallbacks`	同上
`agent.allowedModels`	`agents.defaults.models`	允许模型列表 → models 映射
`agent.modelAliases`	`agents.defaults.models.*.alias`	别名嵌入模型定义
`agent.modelFallbacks`	`agents.defaults.model.fallbacks`	备选模型列表

旧路径	新路径	说明
`identity`	`agents.list[].identity`	身份配置归属到具体 Agent

其他迁移：

旧路径	新路径	说明
`tools.bash`	`tools.exec`	命令行工具重命名
`messages.tts.enabled`	`messages.tts.auto`	布尔值 → 枚举值
`gateway.token`	`gateway.auth.token`	认证配置重组
`routing.transcribeAudio`	`tools.media.audio.models`	音频转录移到媒体工具
`telegram.requireMention`	`channels.telegram.groups."*".requireMention`	同群聊迁移

31.4.3 分阶段迁移（`legacy.migrations.part-1/2/3.ts`）

迁移架构

迁移规则按复杂度和依赖关系拆分为三个阶段，通过 `legacy.migrations.ts` 汇总：

```
// src/config/legacy.migrations.ts

export const LEGACY_CONFIG_MIGRATIONS = [
  ...LEGACY_CONFIG_MIGRATIONS_PART_1,   // 基础迁移（渠道、路由）
  ...LEGACY_CONFIG_MIGRATIONS_PART_2,   // 模型与 Agent 列表
  ...LEGACY_CONFIG_MIGRATIONS_PART_3,   // Agent 默认值与工具
];
```

顺序至关重要——Part 1 先将渠道配置归位，Part 2 才能正确处理 Agent 路由（因为路由迁移可能引用渠道），Part 3 最后清理 `agent.*` 残留到 `agents.defaults` 。

Part 1：基础迁移

Part 1 包含 9 条迁移规则，主要处理字段重命名和位置移动：

规则 1：`bindings.match.provider` → `bindings.match.channel`

早期版本使用 `provider` 来标识消息来源（WhatsApp、Telegram 等），后来统一为 `channel` （渠道）。迁移遍历 `bindings` 数组中的每个条目：

```
{
  id: "bindings.match.provider->bindings.match.channel",
  apply: (raw, changes) => {
    const bindings = Array.isArray(raw.bindings) ? raw.bindings : null;
    if (!bindings) return;

    let touched = false;
    for (const entry of bindings) {
      const match = getRecord(entry?.match);
      if (!match) continue;
      if (typeof match.channel === "string") continue;  // 已有 channel → 跳过
      const provider = typeof match.provider === "string" ? match.provider.trim() : "";
      if (!provider) continue;

      match.channel = provider;
      delete match.provider;
      touched = true;
    }
    if (touched) changes.push("Moved bindings[].match.provider → bindings[].match.channel.");
  },
}
```

规则 5：顶级渠道 → `channels.*`

这是最常见的迁移——将 `whatsapp` 、 `telegram` 等顶级键移入 `channels` 命名空间：

```
{
  id: "providers->channels",
  apply: (raw, changes) => {
    const legacyKeys = ["whatsapp", "telegram", "discord", "slack", "signal", "imessage", "msteams"];
    const legacyEntries = legacyKeys.filter((key) => isRecord(raw[key]));
    if (legacyEntries.length === 0) return;

    const channels = ensureRecord(raw, "channels");
    for (const key of legacyEntries) {
      const legacy = getRecord(raw[key]);
      if (!legacy) continue;

      const channelEntry = ensureRecord(channels, key);
      const hadEntries = Object.keys(channelEntry).length > 0;
      mergeMissing(channelEntry, legacy);   // 只填补缺失值
      delete raw[key];                       // 删除旧键

      changes.push(
        hadEntries
          ? `Merged ${key} → channels.${key}.`
          : `Moved ${key} → channels.${key}.`
      );
    }
  },
}
```

规则 7： `routing.groupChat.requireMention` 扇出

这个迁移比较特殊——单一配置值需要"扇出"到多个渠道的群聊配置中：

```
// 简化逻辑
const applyTo = (key: "whatsapp" | "telegram" | "imessage") => {
  const section = ensureRecord(channels, key);
  const groups = ensureRecord(section, "groups");
  const entry = ensureRecord(groups, "*");           // "*" = 默认规则

  if (entry.requireMention === undefined) {
    entry.requireMention = requireMention;            // 填补缺失
    changes.push(`Moved → channels.${key}.groups."*".requireMention.`);
  }
};

applyTo("whatsapp", { requireExisting: true });  // WhatsApp 需要已有配置
applyTo("telegram");                              // Telegram 总是应用
applyTo("imessage");                              // iMessage 总是应用
```

注意 `requireExisting` 选项——对于 WhatsApp，只有在用户已经配置了 WhatsApp 的情况下才迁移 `requireMention`，避免为未使用的渠道创建空配置。

Part 2：模型与路由迁移

Part 2 包含 3 条迁移规则，处理最复杂的模型配置和 Agent 路由迁移。

模型配置迁移（`agent.model-config-v2`）

这是整个迁移系统中最复杂的规则。早期版本中，模型配置是一组扁平的字段：

```
// 旧格式
{
  "agent": {
    "model": "claude-3-opus",                    // 字符串
    "allowedModels": ["claude-3-opus", "gpt-4"],
    "modelAliases": { "opus": "claude-3-opus" },
    "modelFallbacks": ["gpt-4", "claude-3-sonnet"],
    "imageModel": "dall-e-3",
    "imageModelFallbacks": ["stable-diffusion-xl"]
  }
}
```

需要迁移为结构化格式：

```
// 新格式
{
  "agents": {
    "defaults": {
      "model": {
        "primary": "claude-3-opus",
        "fallbacks": ["gpt-4", "claude-3-sonnet"]
      },
      "imageModel": {
        "primary": "dall-e-3",
        "fallbacks": ["stable-diffusion-xl"]
      },
      "models": {
        "claude-3-opus": { "alias": "opus" },
        "gpt-4": {},
        "claude-3-sonnet": {},
        "dall-e-3": {},
        "stable-diffusion-xl": {}
      }
    }
  }
}
```

迁移过程分为四步：(1) 收集所有涉及的模型名称并在 `models` 映射中创建条目；(2) 将 `modelAliases` 的别名关系嵌入到对应模型的 `alias` 字段；(3) 构建 `model.primary/fallbacks` 和 `imageModel.primary/fallbacks` 结构；(4) 清理旧字段。

Agent 路由迁移（ `routing.agents-v2` ）

将 `routing.agents` 中按 Agent ID 分组的配置迁移到 `agents.list` 数组，同时处理 `mentionPatterns` 、 `groupChat` 、 `sandbox.tools` 等子配置的位置调整。

Part 3：Agent 默认值与工具

Part 3 包含 5 条收尾性质的迁移规则：

`agent.defaults-v2` ： 最终清理

这条规则执行 `agent.*` → `agents.defaults` 的最终迁移，同时将工具相关配置分流到 `tools.*` ：

```
agent.tools.allow   → tools.allow
agent.tools.deny    → tools.deny
agent.elevated      → tools.elevated
agent.bash          → tools.exec
agent.sandbox.tools → tools.sandbox.tools
agent.subagents.tools → tools.subagents.tools
agent.*（其余）       → agents.defaults.*
```

`identity->agents.list` ： 身份配置归属

将顶级 `identity` 移到默认 Agent 的配置中。确定"默认 Agent"使用 `resolveDefaultAgentIdFromRaw` ——它按优先级查找：(1) `agents.list` 中 `default: true` 的条目；(2) `routing.defaultAgentId` ；(3) 列表中的第一个条目；(4) 兜底值 `"main"` 。

其他 Part 3 规则：

规则 ID	迁移内容
`auth.anthropic-claude-cli-mode-oauth`	认证模式 `token` → `oauth`
`tools.bash->tools.exec`	工具重命名
`messages.tts.enabled->auto`	`enabled: true` → `auto: "always"` ， `enabled: false` → `auto: "off"`

迁移规则的设计原则

从源码中可以总结出以下设计原则：

1. **幂等性**：每条规则都先检查目标位置是否已有值。如果已有值，则跳过迁移（或仅删除旧键），确保多次执行结果一致。
2. **只填补不覆盖**： `mergeMissing` 语义确保用户在新位置手动设置的值不会被旧值覆盖。
3. **完整日志**：每条规则执行后都会向 `changes` 数组追加人类可读的变更描述，便于用户了解自动迁移做了什么。
4. **渐进清理**：迁移后如果对象变为空（如 `routing` 下的所有键都被迁走），则删除空对象，保持配置整洁。
5. **顺序依赖**：Part 1 → Part 2 → Part 3 的顺序确保前置迁移完成后再执行依赖它们的迁移。

本节小结

1. **双轨系统**：遗留配置处理分为"检测"（`findLegacyConfigIssues`，生成警告）和"迁移"（`applyLegacyMigrations`，自动修复）两个独立子系统。
2. **深拷贝保护**：迁移在 `structuredClone` 产生的副本上操作，不影响原始输入。
3. **25+ 检测规则**覆盖了渠道迁移、路由拆解、Agent 配置重组、工具重命名等所有历史变更。
4. **三阶段迁移**（Part 1/2/3）按依赖关系排序：基础迁移 → 模型与路由 → Agent 默认值与工具。
5. **幂等设计**确保迁移可以安全地多次执行，不会破坏已经迁移过的配置。
6. **mergeMissing 语义**保证"用户手动设置的新格式配置"不会被旧格式值覆盖。
7. **迁移后验证**（`migrateLegacyConfig`）确保迁移结果通过 Zod 验证，无效则提示手动修复。